Version-II

DEADLOCK DETECTION SIMULATOR

Submitted By:

Kaleem Faryad(51) ,Alyan Jahangir(33),Muhammad Mudasir Hafeez(50)

Session 2023 -2027

(Reg. No.2023-UOK-03651,2023-UOK-03650,2023-UOK-03633)

Submitted To: Mr.Mehtab Mushtaq

Subject: OPERATING SYSTEM

Date Of Submission: Feb 13th,2026

Department of CS&IT

FACULTY OF COMPUTING AND ENGINEERING

UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR

DEADLOCK DETECTION SIMULATOR

(Reg. No. 2023-UOK-03651,2023-UOK-03650,2023-UOK-03633)

Submitted to: Mr. Mehtab Mushtaq

Subject: OPERATING SYSTEM

Session: 2023-2027

Submitted By:

Kaleem Faryad(51) ,Alyan Jahangir(33)Muhammad Mudasir Hafeez(50)

Date of Submission: Feb 13th, 2026

Version-02

DEPARTMENT OF CS & IT

FACULTY OF COMPUTING AND ENGINEERING

UNIVERSITY OF KOTLI AZAD JAMMU AND KASHMIR

Contents

1. REQUIREMENT SPECIFICATION: 4

1.1 Functional Requirements 4

1.2. Non-Functional Requirements 4

2. SOFTWARE ARCHITECTURE DIAGRAM 4

2.1 Process & Resource Interface 4

2.2 Resource Cycle and Detection Engine 5

2.3 Integrated System and Deadlock Analysis 6

1. REQUIREMENT SPECIFICATION:

This section describes the functional and non-functional requirements of the **Deadlock Detection Simulator**. These requirements define the system's core capabilities and the constraints under which the architecture must operate.

1.1 Functional Requirements

- The system shall allow the user to define processes and resources.
- The system shall create a Resource Allocation Graph.
- The system shall identify requests and allocations between processes and resources.
- The system shall detect deadlock by checking for cycles in the graph.
- The system shall display whether a deadlock exists or not.
- The user shall be able to modify resource allocation and recheck deadlock.

1.2. Non-Functional Requirements

- The system shall be easy to use and understand.
- The system shall provide results in a short time.
- The user interface shall be simple and student friendly.
- The system shall provide accurate deadlock detection results.
- The system shall be accessible through a web browser.

2. SOFTWARE ARCHITECTURE DIAGRAM

The Deadlock Detection Simulator is designed using a component-based architecture. The system is divided into multiple independent components, where each component performs a specific function and interacts with other components through well-defined interfaces.

The architecture consists of components such as User Interface, Process Management, Resource Allocation, Cycle Detection Engine, Deadlock Calculator, RAG Generation, Performance Analysis, and Export to Report. These components work together to simulate resource requests and analyze the system for potential deadlocks.

The overall data flow starts from user-defined process/resource counts, moves through allocation and cycle detection, and finally produces calculated results, visual graphs, and exported deadlock reports. This design makes the system modular, easy to understand, and suitable for future enhancements.

2.1 Process & Resource Interface

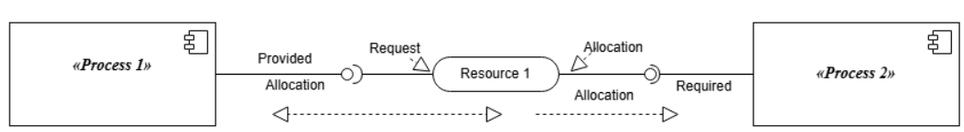

Figure2.1 Process & Resource Interface

Explanation:

This diagram illustrates the fundamental interaction between a single resource and two independent processes. It serves as the basic building block for the larger simulator.

- **Process 1 (Provider):** In this specific interaction, Process 1 acts as the **Provider**. It has been **Allocated** Resource 1, meaning it currently holds the resource to perform its execution.
- **Process 2 (Required):** Process 2 is the **Required** component in this scenario. It has sent a **Request** for Resource 1 but cannot proceed because the resource is already occupied.
- **Resource 1:** This is the shared entity that creates the dependency between the two processes.

2.2 Resource Cycle and Detection Engine

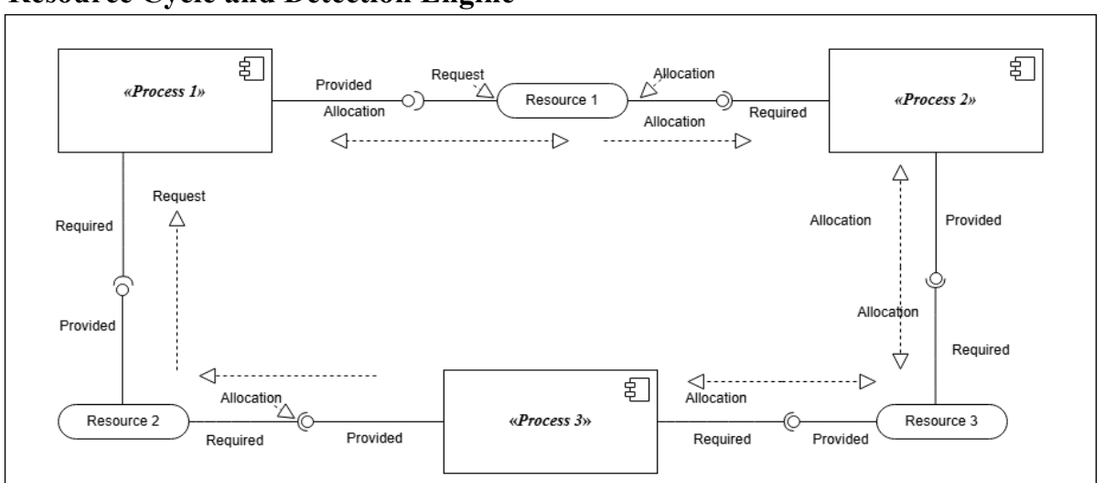

Figure2.2 **Resource Cycle and Detection Engine**

Component Descriptions:

- **Process 1, 2, & 3 (Circular Entities):** In this state, the simulator manages three active processes simultaneously. Each process acts as both a **Provider** (holding a resource) and a **Required** component (requesting a new one), creating a continuous chain of dependency.
- **Resource 1, 2, & 3:** These are the shared entities that facilitate the cycle. The system tracks **Resource 1** as allocated to P1, **Resource 2** as allocated to P3, and **Resource 3** as allocated to P2.
- **Allocation & Request Interface:** The simulator uses dashed lines to represent the flow of dependency, where Process 1 requests Resource 2, Process 3 requests Resource 3, and Process 2 requests Resource 1.

Conclusion:

This diagram illustrates a more advanced architectural state where the **Cycle Detection Engine** identifies a **Circular Wait**. Unlike the simple dependency in the previous stage, this configuration demonstrates a potential deadlock where no process can progress because the resource required by one is held by another in a closed loop. By simulating this multi-process interaction, the system can test its logic for detecting permanent wait states and provide visual feedback to the user regarding system-wide bottlenecks.

2.3 Integrated System and Deadlock Analysis

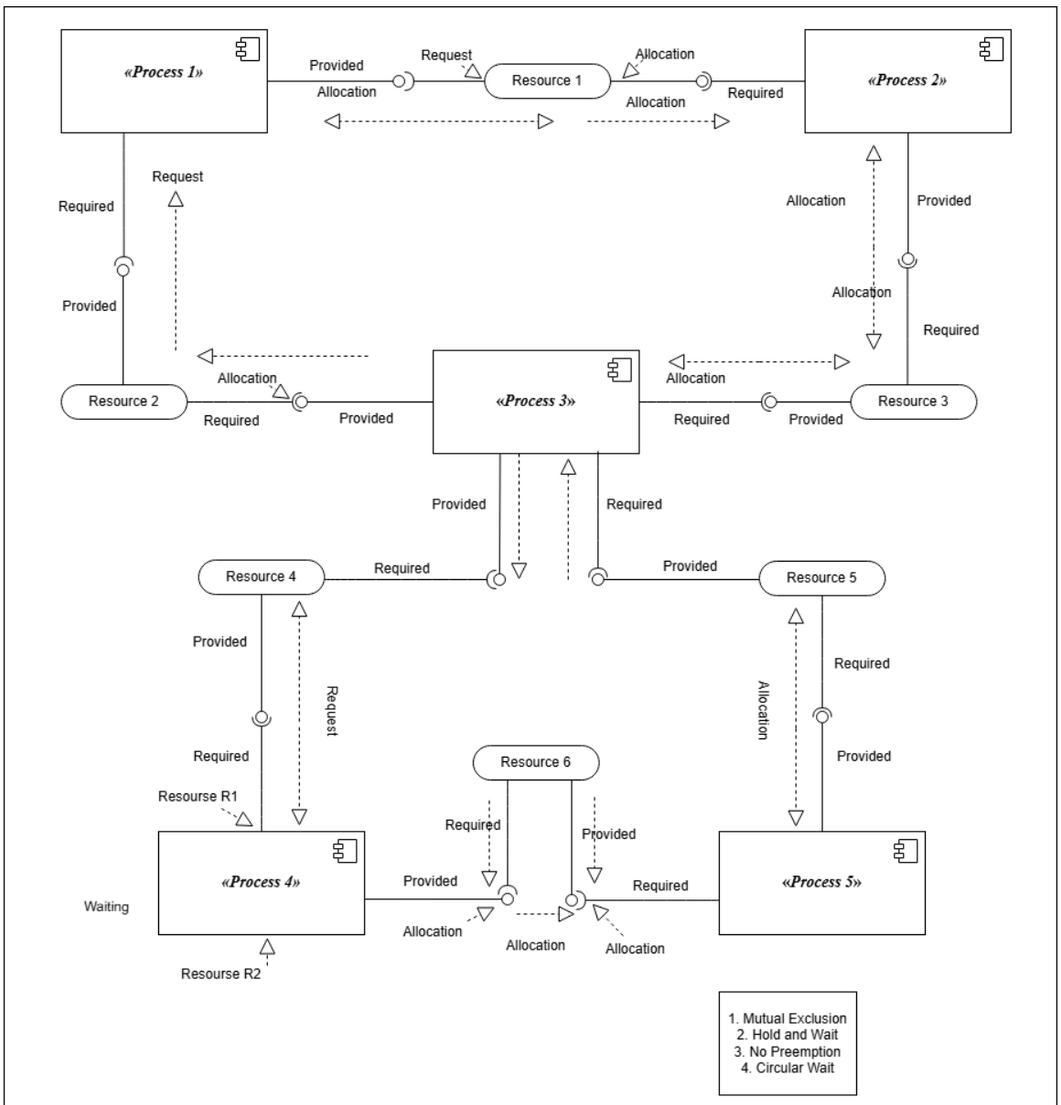

Figure2.3 Resource Cycle and Detection Engine

Component Descriptions:

- **Integrated Process Cluster (P1, P2, P3, P4, P5):** This represents the full set of active processes within the simulation. The system is managing five distinct processes simultaneously, each with unique execution states.
- **Global Resource Repository (R1 to R6):** The architecture tracks all six hardware/software resources. The simulator manages the mapping between these resources and the processes using **Assignment Edges** (Resource → Process) and **Request Edges** (Process → Resource).

- **Analysis Output (Waiting Status):** The "Waiting" labels on processes (such as P4) represent the output of the **Performance Analysis** module, indicating which processes are currently blocked due to the identified deadlock.

Conclusion:

This diagram illustrates the complete data flow of the simulator. It shows how the system moves from individual user inputs to a global analysis of system stability. By combining the cycle detection from earlier phases with the full resource grid, the simulator can accurately identify where the system has halted. This integrated view reflects a robust, component-based software design, demonstrating how the **Simulation Engine** and **Deadlock Calculator** work together to provide a high-level overview of process-resource interactions.